



KONYA

FOOD & AGRICULTURE

UNIVERSITY

16-bit PROCESSOR VERILOG REPORT

Semester: 2025–2026 Fall

Course: COMP3005 - Computer Organization

Project: 16-bit RISC Pipelined Processor Design

Date: January 14, 2026

Preparers Of The Project:

Muhammed Melih ÇAM - 222010020116

Şeref Samet ÜMÜTLÜ - 212010020118

Mert KIRÇİÇEK - 232010020044

Zeynep Sude BERBER - 212010020075

1. Introduction

This report documents the second phase of the 16-bit RISC Processor design project: the Verilog Implementation Phase. Following the architectural definition and visual validation performed in Phase 1 (Logisim Design), this phase focuses on translating the microarchitectural concepts into a synthesizable Register Transfer Level (RTL) description using Verilog HDL.

The primary objective of this implementation is to validate the timing, control logic, and data flow of the 5-stage pipeline (IF, ID, EX, MEM, WB) in a hardware description environment. Unlike the abstract visual simulation in Logisim, the Verilog implementation enforces strict timing constraints, precise signal propagation, and realistic memory modeling.

Key achievements in this phase include:

- Full RTL coding of the 5-stage pipeline.
- Implementation of advanced hazard handling (Forwarding, Stalling, and Flushing).
- Cycle-accurate verification using a custom testbench and machine code scenarios.

2. Methodology & Design Hierarchy

The design adopts a modular hierarchy to ensure maintainability and debuggability. The system is encapsulated within a top-level module that acts as a wrapper for the processor core and memory units.

2.1 Top-Level Module (cpu_top.v)

The `cpu_top` module serves as the simulation boundary. It instantiates the main processor core (`PipelineProcessor`) and exposes internal debug signals (such as Program Counter, Register R1/R2 values, and specific Memory addresses) to the testbench. This structure allows for "white-box" testing, where internal states can be monitored without altering the core logic.

2.2 The Core: PipelineProcessor (PipelineProcessor.v)

This is the heart of the design. It structurally connects the five pipeline stages. Instead of a single monolithic block, the stages are separated by pipeline registers (e.g., if_id, id_ex, ex_mem, mem_wb). These registers are clocked synchronously, ensuring that data advances through the pipeline one stage per clock cycle.

3. Detailed Pipeline Stage Implementation

3.1 Instruction Fetch (IF) Stage

- **Module:** InstructionMemory.v, ControlHazardUnit.v
- **Functionality:** The Program Counter (PC) drives the address bus of the Instruction Memory. Under normal operation, the PC increments by 2 (byte-addressable, 16-bit words).
- **Control Flow Logic:** The ControlHazardUnit plays a critical role here. It determines the pc_next value. If a branch is taken (resolved in the EX stage), the sequential PC is overridden by the branch_target.
- **RTL Detail:** The Instruction Memory is initialized using \$readmemb("machine_code.txt", ...) to load the binary program during simulation start-up.

3.2 Instruction Decode (ID) Stage

- **Modules:** RegisterFile.v, ControlUnit.v, HazardUnit.v
- **Decoding:** The 16-bit instruction is split into Opcode, Rs, Rt, Rd, and Immediate fields.
- **Internal Forwarding:** The RegisterFile includes a specific improvement over standard designs. If a read operation addresses the same register currently being written to in the WB stage, the module bypasses the memory array and

outputs the write_data directly. This resolves "Read-After-Write" hazards within the register file itself.

- **Stall Generation:** The HazardUnit operates in this stage. It detects Load-Use hazards (where an instruction needs data currently being loaded from memory). If detected, it asserts a stall signal, freezing the PC and IF/ID registers and inserting a bubble into ID/EX.

3.3 Execute (EX) Stage

- **Modules:** ALU.v, ForwardingUnit.v
- **ALU Operations:** The ALU handles arithmetic (ADD, SUB), logical (AND, OR), comparison (SLT), and shifting (SLL, SRL) operations. Crucially, branch outcomes (Zero flag) are calculated here.
- **Forwarding Logic:** The ForwardingUnit monitors the pipeline. If an instruction in EX needs data computed by previous instructions still in MEM or WB stages, the unit overrides the register file output with the forwarded data. This minimizes stalls and maintains throughput.

3.4 Memory (MEM) Stage

- **Module:** DataMemory.v
- **Operation:** This stage handles Load (LW) and Store (SW) operations. The memory is implemented as a 256-word array.
- **Addressing:** Since the architecture is byte-addressable but the memory array is word-organized, the effective index is calculated as `addr[9:1]`.

3.5 Write Back (WB) Stage

- **Logic:** The final stage selects between ALU results or Memory Load data (based on MemToReg signal) and writes the result back to the write_reg in the Register File.

4. Hazard Management & Resolution

One of the most complex aspects of the Verilog implementation was ensuring data integrity during pipelined execution.

4.1 Data Hazards (Forwarding)

Data hazards occur when an instruction depends on the result of a preceding instruction that hasn't been written back yet.

- **Implementation:** The ForwardingUnit.v compares rs/rt of the current instruction (ID/EX) with the destination registers of the pipeline (EX/MEM and MEM/WB).
- **Result:** Data is forwarded from the ALU output or Memory output directly to the ALU input, eliminating the need for NOPs in most cases.

4.2 Control Hazards (Flushing)

Branching changes the flow of execution. Since the decision is made in the EX stage, two incorrect instructions (in IF and ID) have already been fetched.

- **Implementation:** The ControlHazardUnit.v detects branch_taken. It asserts if_id_flush and id_ex_flush.
- **Effect:** The valid bits or control signals of the instructions in the "wrong path" are cleared (converted to NOPs/Bubbles), preventing them from altering the processor state.

5. Simulation & Verification Results

The functional verification of the 16-bit RISC processor was conducted using the Xilinx Vivado Simulator. A comprehensive testing strategy was adopted to validate not only the execution of individual instructions but also the correct operation of the pipeline mechanics, including hazard resolution (forwarding, stalling, flushing) and memory interactions.

5.1 Test Strategy and Environment

The verification process involved a custom testbench module (TestBench.v) designed to simulate the processor under various operating conditions. The test environment provided a stable clock signal (10ns period) and reset logic to initialize the system.

Instead of relying on a single test case, the processor was subjected to multiple machine code sequences targeting different architectural features:

- **Arithmetic & Logic Operations:** Verified the ALU's ability to perform calculations (ADD, SUB, AND, OR, Shifts) and update flags correctly.
- **Memory Access:** Tested LW and SW instructions to ensure data integrity during memory reads and writes, including the handling of Load-Use hazards.
- **Control Flow:** Validated correct branching (BEQ, BNE) and jumping (J, JAL, JR) behavior, ensuring that the ControlHazardUnit correctly flushes the pipeline when the flow of execution changes.
- **Data Forwarding:** Specifically tested sequences with data dependencies (e.g., Read-After-Write) to confirm that the ForwardingUnit bypasses the register file to prevent unnecessary stalls.

5.2 Waveform Analysis

The behavioral simulation results confirm the cycle-accurate operation of the processor. Figure 1 illustrates a snapshot of the simulation waveform, demonstrating the active pipeline stages and signal propagation.

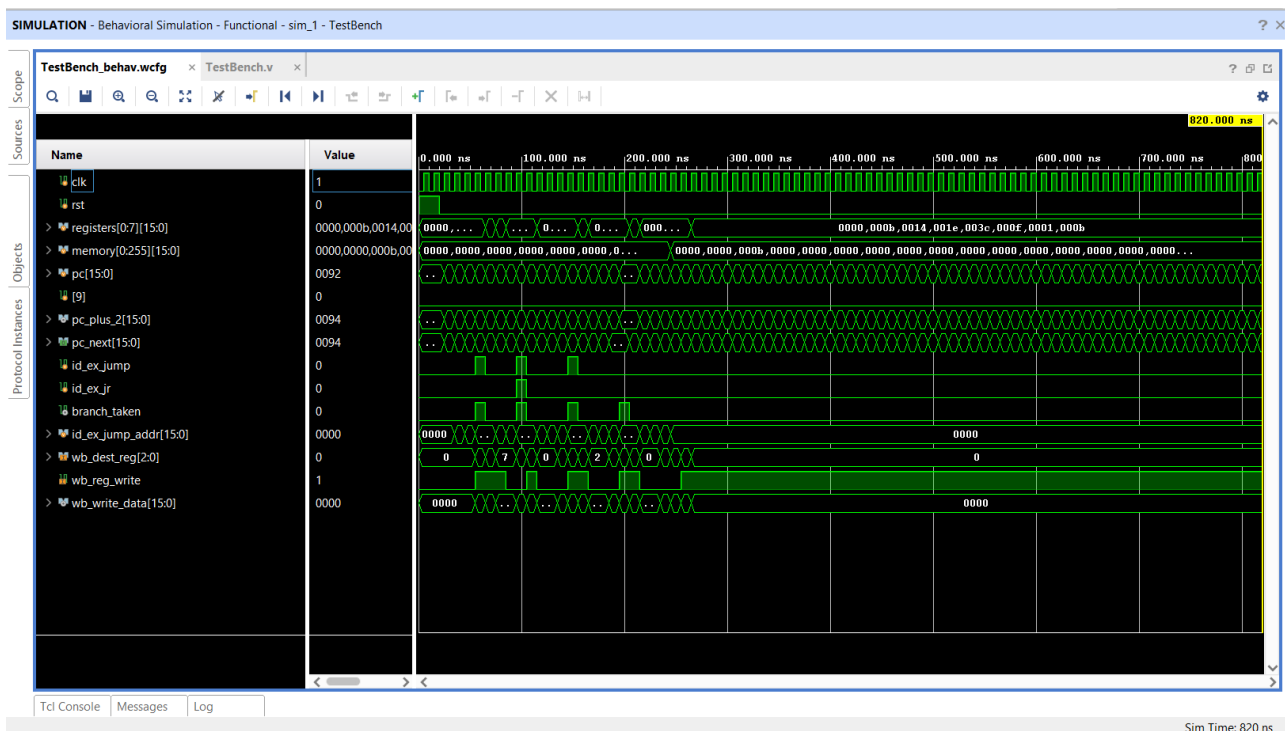


Figure 1: Timing diagram from the Vivado simulation environment showing the processor signals during a comprehensive test run.

Analysis of the Simulation Output: The waveform in Figure 1 provides visual evidence of the processor's stability and functionality:

1. **Clock and Reset:** The system operates synchronously with a clean clock signal (clk). The reset sequence (rst) correctly initializes the Program Counter and internal registers to a known state before execution begins.
2. **Instruction Fetch & PC Update:** The Program Counter (pc) signals demonstrate sequential incrementation (PC+2) during normal operation, and immediate updates during control-flow instructions, validating the fetch logic.
3. **Pipeline Flow & Register Updates:** Activity in the pipeline registers (visible as signal transitions in id_ex, wb_dest_reg, etc.) confirms that instructions are successfully decoding, executing, and reaching the Write-Back stage. The dynamic changes in control signals (such as id_ex_jump or branch_taken) indicate that the control unit is actively responding to the instruction stream.
4. **Hazard Management:** The seamless flow of instructions without undefined states ('X') indicates that hazard detection units are correctly managing pipeline dependencies, inserting stalls or flushes only when architecturally necessary.

6. Conclusion

The successful completion of the Verilog Implementation Phase marks a critical milestone in the development of the 16-bit RISC pipelined processor. By transitioning from the high-level visual models of the previous phase to a concrete Register Transfer Level (RTL) description, this project has validated the architectural feasibility and timing characteristics of the design under strict hardware constraints.

The core achievement of this phase is the realization of a fully functional 5-stage pipeline (IF, ID, EX, MEM, WB) that adheres strictly to the defined Instruction Set Architecture. The implementation of modular components, such as the ALU, Control Unit, and Register File, demonstrated the effectiveness of a hierarchical design approach in managing system complexity. This modularity not only facilitated parallel development but also simplified the debugging process during integration.

Crucially, the operational integrity of the pipeline was secured through the rigorous implementation of hazard management protocols. The integration of a Forwarding Unit successfully resolved data dependencies (Read-After-Write hazards), significantly minimizing pipeline stalls and optimizing instruction throughput. Furthermore, the Hazard Detection and Control Hazard units ensured the correct handling of load-use dependencies and control-flow changes (branches and jumps) via precise stalling and pipeline flushing mechanisms.

Final verification, conducted through cycle-accurate simulations in the Xilinx Vivado environment, confirmed that the processor operates reliably across a variety of test vectors, including arithmetic calculations, memory operations, and complex branching scenarios. The waveform analyses demonstrated correct signal propagation, register updates, and memory interactions, proving that the developed RTL code provides a faithful, robust, and logically correct implementation of the targeted 16-bit RISC architecture.